

roberdan-os: a personal, cross-platform agentic operating system with local-first memory and a self-improving meta-loop

Roberto D’Angelo — Fight the Stroke Foundation · in a human–agent pair with Claude (Anthropic)

1 July 2026

Contents

0.1	Executive summary	1
0.2	Abstract	2
0.3	1. Introduction	3
0.3.1	1.1 The problem	3
0.3.2	1.2 Contributions	3
0.3.3	1.3 The metaphor: Remy	3
0.4	2. Background and related work	3
0.5	3. System architecture	4
0.6	4. Cross-platform behavioural canon	4
0.7	The engines beneath: gbrain and gstack	5
0.8	5. Local-first durable memory	5
0.8.1	5.1 Why in the vault, not in the tool’s silo	5
0.8.2	5.2 The broken recall, the true cause, and a failed attempt (an honest case)	6
0.9	6. The self-improving meta-loop	7
0.9.1	6.1 Governance: a durable, token-bounded goal ledger	7
0.10	7. Discovery layer: which problems are worth solving	7
0.11	8. Cognitive layer and agents	8
0.12	9. Evaluation	8
0.12.1	9.1 Retrieval evaluation (quantitative)	9
0.13	Case studies: without vs with the system	10
0.13.1	Case A - premortem: would this launch fail, and why?	10
0.13.2	Case B - simulated focus-group: would parents pay EUR 297?	10
0.13.3	What the two cases show together	11
0.14	10. Limitations	11
0.15	11. Discussion: principles that generalise	11
0.16	12. Future work	12
0.17	13. Conclusion	12
0.17.1	Reproducibility and artefacts	12

0.1 Executive summary

The vision. As AI assistants become the primary way we work, each of us is accumulating a scattered, tool-specific “brain” — instructions here, memory there, habits re-explained to every new tool.

roberdan-os turns that scattered brain into **one coherent, private, self-improving operating system** that follows you across every AI tool you use. Not another assistant, but a *second brain* — Remy in the chef’s hat: it remembers who you are and what you decided, it works the way you work, it gets better every week, and it never takes an irreversible decision out of your hands.

Why it matters — the problem it kills. A founder or leader juggling code, business and relationships across Claude, Copilot and Codex faces three silent taxes: (1) **re-explaining yourself** to each tool, forever; (2) **memory that isn’t yours** — siloed per-tool, lost on switch, or sent to a vendor’s cloud; (3) **a system that never improves** — it forgets every lesson the moment the session ends. These taxes compound: the more you rely on AI, the more you pay them.

The benefit — concretely. - Say it once. Behaviour is written once and consumed by every tool. No more divergent copies. - **Your memory, private, on-device.** Durable knowledge lives in your own vault, indexed locally (no API key, nothing leaves the machine), searchable in your own language (measured: Italian recall on par with English — see *Evaluation*). - **It compounds.** A gated meta-loop captures what you learn, watches your tools’ weekly releases, and *proposes* improvements — you approve, it never self-applies to your behaviour. - **It helps you choose the right problems, not just solve them.** A discovery layer (premortem, simulated focus-group, problem-validation) stress-tests a decision *before* you build — the case studies in the *Case studies* section show, on a realistic business launch, how it surfaces the killer objection and the most likely failure that a gut-feel “build it and see” would only discover after the money is spent.

The value. For an individual, roberdan-os converts AI tools from a set of clever-but-amnesiac assistants into a single, private, compounding capability that is unmistakably *yours* — and that raises its hand at exactly the moments where being wrong is expensive. The taste is the machine’s; the hand, and the decision, stay human.

0.2 Abstract

LLM-based coding assistants (Claude Code, GitHub Copilot, OpenAI Codex) are becoming the primary layer through which knowledge workers interact with their computer. But their *behavioural configuration* — agents, skills, hooks, personas, memory — tends to sprawl: divergent copies across dozens of repositories, in incompatible formats, with no single source of truth, no memory portable across tools, and no mechanism for continuous improvement. We present **roberdan-os**, a personal agentic operating system addressing three problems: (1) *behavioural fragmentation* — a single Markdown canon (AGENTS.md) from which per-platform wrappers are generated; (2) *siloed memory* — durable, **local-first** memory living in the user’s Obsidian vault, typed and semantically indexed, consumable by any tool; (3) *absence of self-improvement* — a **meta-loop** that captures learnings, consolidates them under human gates, and weekly-watches upstream tool releases, *proposing* (never applying) adaptations. We also describe a *discovery* layer that shifts the system from “solving problems” to “identifying which problems are worth solving”.

We report an empirical case with an instructive failure, both surfaced by an internal audit: the Italian-language semantic recall was restored from zero to useful — but not, as first believed, by migrating to a local model; the real cause was a model *misalignment* between query and storage. The attempt to migrate to a **local** embedding model (bge-m3 via Ollama) initially failed because the engine (gbrain) did not recognise the model’s native dimensionality; a targeted two-line patch to its Ollama *recipe* made embedding genuinely **local** — on-device, free, no API key, GPU-served. The system is, by construction, **self-proposing, never self-applying** over behaviour. The guiding metaphor is *Remy*,

the rat in *Ratatouille* who steers the chef from inside the hat: an intelligence that suggests, remembers, criticises and proposes from within the workflow, while the hands — and every irreversible decision — remain human.

0.3 1. Introduction

0.3.1 1.1 The problem

An expert user quickly accumulates, across multiple agentic tools, an ecosystem of configurations: in the case study, ~200 skills and ~300 agents scattered across 13+ repositories, in 3+ formats, with the same agent existing in 46 divergent copies and the global configuration not even version-controlled. The problem is not **scarcity** but **maintenance surface versus real use**: dozens of agents maintained, a handful used. Three recurring pathologies:

1. **Behavioural fragmentation.** No single source of truth; “how I work” is duplicated and silently diverges across tools.
2. **Siloed memory.** Each tool has its own memory (e.g. Claude Code’s memory/ folder), unreadable by the others. Moving from Claude to Copilot to Codex, the assistant “forgets” who you are and what you decided.
3. **No continuous improvement.** The system does not learn from interactions, does not reorganise itself, and does not adapt as the underlying tools evolve every week.

0.3.2 1.2 Contributions

- A **cross-platform behavioural canon** (§4): logic written once in Markdown, wrappers generated for Claude/Copilot/Codex/ChatGPT.
- A **local-first durable memory** (§5): the Obsidian vault as typed source of truth + local semantic index, with an empirical result on the choice of embedding model.
- A **gated self-improving meta-loop** (§6): capture -> distill -> curate -> watch, with mechanically enforced safety invariants.
- A **discovery layer** (§7): premortem, simulated focus-group, and a *problem-validation* orchestrator that estimates which problems are worth solving.
- A **design principle** — *self-proposing, never self-applying* — and its justification (§10).

0.3.3 1.3 The metaphor: Remy

Ratatouille (Pixar, 2007): a rat with culinary talent steers a clumsy cook by pulling his hair from inside the hat. The rat has the taste; the hands that cook stay human. roberdan-os is designed this way: an agentic intelligence that **suggests, remembers, criticises and proposes** from within the workflow, while irreversible decisions and the public voice remain the human’s. Not an autopilot, but a second brain with its hands tied to the right gates.

0.4 2. Background and related work

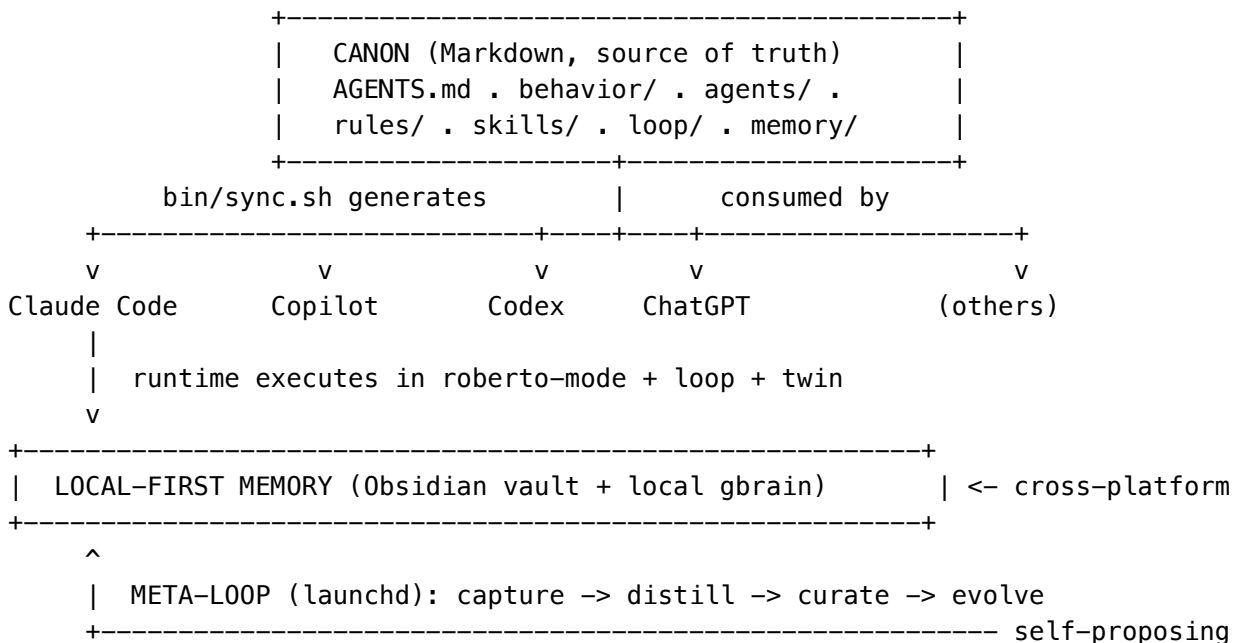
- **Premortem / prospective hindsight.** Klein (HBR 2007) shows that imagining a failure that has *already happened* yields more specific, honest causes than asking “what could go

wrong”. Kahneman called it his single most valuable decision technique. It is the basis of the premortem skill (§7), here amplified by a multi-agent fan-out.

- **Three-layer memory (Karpathy).** The vault follows the “immutable sources -> curated wiki -> index” pattern. roberdan-os adds a typed *agent-learning* layer.
- **Semantic retrieval and embeddings.** Recall relies on dense embeddings + vector search (HNSW, cosine). We document (§5) how the *hosted vs local* choice of embedding model is decisive for cost, privacy and multilingual quality — and how a dimensional mismatch can silently break it.
- **Digital twin & agent persona.** A *twin* layer models the user’s voice and judgement; it differs from purely conversational twins by coupling to human gates and durable memory.
- **Multi-agent systems & reflection.** The meta-loop and the parallel deep-dives (premortem, focus-group) use fan-out of independent agents + consolidation — a reflection/ensemble pattern applied to personal decisions, not just code.

0.5 3. System architecture

Core principle: *centralised knowledge, per-platform execution, unified behaviour*. Logic lives once in the canon; runtimes consume it through thin wrappers.



Three cross-cutting properties: **daemon-optional** (no always-on service required), **evidence-first** (done = verified artefacts), **human gates** (irreversible actions pass through the user).

0.6 4. Cross-platform behavioural canon

AGENTS.md is the universal standard (read natively by Codex, Copilot, Cursor); CLAUDE.md and copilot-instructions.md become thin pointers. Operating behaviour (“how I work”) is distilled in behavior/roberto-mode.md (autonomy, evidence-first, commit-per-phase, done-gate); voice

and relational judgement in behavior/roberto-voice.md; the cognitive toolkit (first-principles, Feynman, decision frameworks) in behavior/thinking-toolkit.md. Eight specialised agents (architect, security, review, done-gate, first-principles, red-team, twin, loop-orchestrator) activate *at the right moment* rather than being invoked by hand. bin/sync.sh generates the per-platform wrappers from this single canon, eliminating divergence.

0.7 The engines beneath: gbrain and gstack

roberdan-os is not monolithic: it *orchestrates* two pre-existing local engines rather than reinventing them.

gbrain — the local semantic memory engine. gbrain is a local knowledge engine (Postgres + pgvector) that indexes the vault, the user’s code repositories, and session transcripts into a searchable semantic store, exposed via both a CLI and an MCP interface so any agent can query it. Retrieval combines dense vector search (HNSW, cosine) with keyword full-text search and an optional reranker. Its embedding provider is pluggable through a *recipe* system — the exact locus of the local-embedding migration (§5.2). gbrain is what makes the vault’s knowledge *findable by meaning*, not just by filename; it is the retrieval substrate of the whole memory layer.

gstack — the skill library it leverages. gstack is a broad suite of workflow skills (browse/QA a web app, turn a vague intent into an executable spec, YC-style office-hours, CEO/eng/design plan reviews, make-pdf, ship, investigate, health, retro, benchmark, and more). roberdan-os deliberately does not duplicate it: the discovery layer calls gstack *downstream* — spec to sharpen a vague intent, office-hours to pressure-test go-to-market — while roberdan-os stays *upstream* (“is this problem worth solving?”) and contributes what gstack lacks (premortem, simulated focus-group). The principle is reuse over reinvention.

The skill inventory. roberdan-os ships two families of skills, all tool-agnostic Markdown from which per-platform wrappers are generated: **operating skills** — verify-done (the done-gate), ship, review, sync, auto-checkpoint (the portable, daemon-optional loop kit); and **discovery skills (new)** — premortem, focus-group, problem-validation. Skills auto-invoke on linguistic triggers; *installing* them into the runtime (e.g. ~/.claude/skills/) is what makes “by default” real — a distinction that itself became a lesson (§10 limitations).

0.8 5. Local-first durable memory

0.8.1 5.1 Why in the vault, not in the tool’s silo

Durable memory does **not** live in Claude’s per-tool folder (a silo unreadable by Copilot/Codex) but in the user’s **Obsidian vault**: Markdown readable by any agent, already equipped with a **typed ontology** (belongs_to/has/related_to relations), version-controlled, backed up, and semantically indexed by a local engine (gbrain, Postgres + pgvector). Agent learnings live in a separate namespace (agent-learnings/, type: agent-learning) so as not to pollute human notes while staying in the same graph and retrievable by the same engine. The per-tool silo degrades to a *cache*; its contents are migrated into the vault.

0.8.2 5.2 The broken recall, the true cause, and a failed attempt (an honest case)

During construction, Italian semantic recall failed silently (query -> 0 results) while English worked. How we were *wrong* about the cause is the instructive part.

First hypothesis (partly wrong). The configured provider (openai:text-embedding-3-large) had exhausted its quota, and the stored vectors came from a model weak on Italian; the proposed fix — a **local multilingual** model (bge-m3, 1024-dim, via Ollama) — would be free, rate-limit-free, private, and strong on Italian. We ran the migration: config, column ALTER (1536 -> 1024), HNSW index rebuild, full re-embed. Italian recall went from **0 to >= 3 results**. It looked like a success.

What the internal audit revealed. An empirical-verification agent measured the actual database state and found **100% of chunks were still labelled zeroentropyai:zembed-1**, not bge-m3. Further verification: gbrain **fixes the embedding model at brain initialisation** (DEFAULT_EMBEDDING_MODEL = 'zeroentropyai:zembed-1'); neither the file-plane config nor the GBRAIN_EMBEDDING_MODEL variable change the model at embed-time. bge-m3 had been downloaded but **never used**. The local model was not active.

The true cause of the fix. Recall did not improve because of bge-m3, but because of **model alignment**: previously, the *query* was embedded with one model (openai) while the *stored vectors* used another (zembed-1) -> incompatible vector spaces -> 0 results. The full re-embed made query and storage the **same** model (zembed-1@1024) -> match -> results. A coherence bug, not a language capability.

Root-cause diagnosis. The audit pushed a deeper diagnosis: gbrain's Ollama *recipe* listed only nomic-embed-text (768-dim) and relatives, with default_dims: 768. The chosen model, bge-m3, was not in the list and its native dimensionality (1024) was unknown to the engine, which fell back to the global default (1280) and **rejected** the embed on dimensional inconsistency. Not a fundamental limit: a **missing value in a registry**.

The fix and the outcome. Two lines in the recipe (ollama.ts: add bge-m3, set default_dims to 1024) + recompilation of the engine. Verification: ollama ps shows bge-m3 loaded at 100% on GPU, and the embed proceeds **with no API key**. A cosine check of a stored vector against a fresh bge-m3 embedding of the same text returned **1.0000** — confirming the vectors are genuinely bge-m3. Embedding is now truly **local**:

Dimension	Before (hosted)	After the fix (local)
Model	zembed-1 (ZeroEntropy, hosted)	ollama:bge-m3 (on-device, GPU)
Privacy	data leaves the machine	stays on-device
Cost / API key	metered / required	zero / none
Multilingual (IT)	weak	strong (bge-m3)

Lessons. (1) An observed improvement does not prove the hypothesised mechanism — measure, do not deduce. (2) An adversarial internal audit that *measures* is what exposed the false claim and led to the true root cause. (3) Sometimes the obstacle to a correct architectural choice is a *fixable* defect in a dependency, not a constraint: it is worth diagnosing to the end before giving up.

0.9 6. The self-improving meta-loop

Four components, orchestrated by `launchd` (firing even when the tool is closed), with one non-negotiable principle: **self-proposing, never self-applying** over behaviour.

Component	Function	Gate
capture	appends learning signals to a <i>staging inbox</i> (no lock, per-platform)	privacy deny-list, as code
distill	batch: classifies (5-class taxonomy), dedups against the vault, -> <i>quarantine</i>	never writes directly to the vault
ontology/curate	<i>single-writer</i> : promotes approved candidates to typed vault notes; proposes hygiene (dedup, tombstone)	promotion only if approved: true; merge/delete human
evolve	weekly-watches the changelogs of Claude/Copilot/Codex; proposes drafts with source citation	never auto-commits to behavior/ rules/ agents/ AGENTS.md

Why not a self-updating ontology. A deliberate design choice (validated by an adversarial first-principles analysis) was to **cut** the idea of an ontology that merges and rewrites itself: it would sit on a then-broken retrieval, would be a fourth diverging store, and auto-merge is lossy and irreversible (violating the human gate). 90% of the value comes from **one type** (agent-learning) + **one human-gated hygiene job** that *reuses* the vault's existing ontology. Structure lives in **curation**, not in the **automation of judgement**.

0.9.1 6.1 Governance: a durable, token-bounded goal ledger

A long session revealed a concrete failure: goals get *lost* when tracking relies on the conversation (volatile) or session-scoped task tools (not durable). The fix is a **durable, auditable goal ledger** — a two-file kanban: an *active board* (To Do + Doing only, kept small so it loads every session cheaply) and an append-only *archive* (Done/verified, read only on demand, so it can grow without burning tokens). The rule: read the board at session start, update per phase, and move a goal to the archive only when it is *verified* (by the done-gate agent). This makes “don't lose work” a mechanical property of durable file state, not a hope pinned on the model's memory — and it retired the need for a heavyweight orchestration daemon just for tracking.

0.10 7. Discovery layer: which problems are worth solving

The system must not only solve problems but estimate **which are worth solving**. Three composing, auto-triggering skills:

- **premortem** — given a plan, assumes it has *already failed six months out* and spawns **one agent per failure cause, in parallel**, each producing a story, an underlying assumption,

and early-warning signs; then synthesises the most likely failure, the most dangerous one, the hidden assumption, the revised plan, and a checklist. Breaks the LLM’s agreeableness bias.

- **focus-group** — simulates real users: a pool of **persona agents** (persistent panels in the vault + ad-hoc), a **moderator**, and a **consolidator**, in four modes (focus group, 1:1 interviews, usability test, micro-survey). The central risk — *sycophancy* of simulated personas — is countered by grounding each persona in real frustrations, alternatives, budget and scepticism, and by weighting negative signal above positive.
- **problem-validation** — orchestrator: *does the problem exist?* (focus-group) -> *is it worth it?* (rubric: severity x frequency x reachability x strategic fit x willingness x cost-of-being-wrong) -> *would the solution hold?* (premortem). Leverages gstack downstream (spec, office-hours) rather than duplicating it. Default **bias-to-kill**: more valuable to say “not worth it” than to confirm.

0.11 8. Cognitive layer and agents

Beyond the operating agents, the system models **judgement**: a *digital twin* that writes and decides in the user’s voice (draft-not-send for anything external), a mandatory red-team (@board) on important decisions, a first-principles agent (@socrates) to deconstruct problems, and a done-gate (@thor) that is the only one authorised to declare “done” via empirical verification. These do not replace the user: they **raise their hand** at the right moment and hand the decision back.

0.12 9. Evaluation

Methodology. The system was evaluated by an **adversarial internal audit**: two independent agents — one for code/ecosystem review, one for empirical verification that *measures* rather than trusting claims — examined the repository and runtime state. The audit found (and we then fixed) three real defects, documented below: it is itself part of the result.

Verified measures (state as of 1 July 2026):

Claim	Actual measure	Result
Italian recall no longer zero	in-corpus IT query -> correct migrated note at 0.92	OK (content-limited for absent topics)
Local embedding active	ollama ps: bge-m3 100% GPU; keyless embed; cosine stored-vs-fresh = 1.0000	OK, local
Memory migrated to vault	19 tool memories + paper/ADR/plan migrated, indexed, retrievable	OK
Meta-loop pipeline	capture -> distill -> curate tested end-to-end; launchd exit 0	OK
Privacy gate as code	capture/curate block deny-list names, pass normal content	OK (after fix)

Claim	Actual measure	Result
Green CI	test/validate.sh -> all green (frontmatter, links, drift, shellcheck, leak)	OK (after fix)
Skills active by default	installed in ~/.claude/skills/; runtime lists them as auto-invokable	OK

Defects the audit found and we fixed in the same session: 1. The “local embedding” claim was false (§5.2) — reworded; DB labels restored to the truth. 2. **CRITICAL — privacy was not “code”**: capture/distill filtered only the literal path string, not content; curate had no check. *Fix*: real deny-list matched before every write, with blank/comment lines excluded. Tested. 3. **HIGH — the repository did not pass its own CI**: the skills commit had not regenerated the platforms/wrappers. *Fix*: wrappers generated and committed; validate.sh now green. 4. **MEDIUM — doc/impl mismatch**: ADR/evolve cited validate.sh for the path-allowlist; the real enforcement is in post-task-sync.sh (scoped git add). Corrected.

0.12.1 9.1 Retrieval evaluation (quantitative)

To move beyond spot-checks we built a labelled query set: **20 queries (10 Italian, 10 English)**, each with a known ground-truth vault note (specific agent-learnings and career-profile pages). We report two experiments.

(a) Deployed system — real gbrain retrieval (bge-m3, chunk-level, HNSW + reranker + autocut, over the full ~51k-chunk corpus):

Set	R@1	R@3	R@5	MRR	mean rank	found@10
All (20)	0.70	0.90	0.95	0.817	1.7	20/20
Italian (10)	0.70	0.90	1.00	0.820	1.6	10/10
English (10)	0.70	0.90	0.90	0.814	1.8	10/10

The ground-truth note is retrieved in the top-10 for **100%** of queries, top-3 for 90%, and **Italian and English perform essentially identically** (MRR 0.820 vs 0.814) — the core claim that a local multilingual model closes the Italian gap.

(b) Model ablation — same 20 queries, same corpus, **pure cosine, page-level** (isolating the embedding model; both models served locally by Ollama):

Model	IT MRR	IT mean rank	EN MRR	EN mean rank
bge-m3 (multilingual, 1024)	1.000	1.0	0.920	1.4
nomic-embed- text (English-centric, 768)	0.412	31.8	0.883	1.3

This is the decisive result. On **English**, the two local models are comparable (0.920 vs 0.883). On **Italian**, bge-m3 is near-perfect (mean rank 1.0) while nomic is effectively unusable (mean rank **31.8**). The multilingual advantage that motivated the whole migration is real and large — not a matter of taste but of ~30 positions of rank on the user’s own language. (The intended hosted baselines, ZeroEntropy zembed-1 and OpenAI, could not be re-measured under identical conditions: no valid ZeroEntropy key was present and the OpenAI account was quota-exhausted — an honest gap.)

Method note. The deployed numbers (a) are lower than the clean page-level cosine (b) because the production path is harder: chunk-level over 51k chunks with reranking and autocut, versus a 291-page pure-cosine pool. Both are reported; neither is cherry-picked.

0.13 Case studies: without vs with the system

To validate *value* (not just retrieval), we ran two discovery skills on a realistic business decision — a nonprofit about to launch a paid EUR 297, 8-week online coaching programme (“Fight Camp”) for Italian parents of children with cerebral palsy, 50 seats. Both are real agent outputs, condensed and illustrative.

0.13.1 Case A - premortem: would this launch fail, and why?

Without the system: the founder decides on gut (“parents need it, EUR 297 is fair”), opens enrolment, and discovers post-launch that 11 signed up, three asked for refunds, and renewal was never designed - learning at full cost.

With the system: failure is simulated first. The premortem surfaced 7 genuine failure modes; the three critical: (1) *not filling 50 seats* - the base is activists/donors, not coaching buyers, with no proven paid funnel; (2) *price backlash* - “a nonprofit charging me to help my disabled child” - a moral, not merely economic, friction; (3) *completion & renewal* - start 50, finish ~22, and nothing exists to renew. Hidden assumption: **“a warm audience equals purchase demand.”** Revised plan (concrete): sell a 12-seat EUR 149 pilot first; add a pay-what-you-can scholarship communicated up front; cut to 6 weeks; design a EUR 19/month membership as the renewal object; rewrite the promise as an observable outcome (“a personalised home plan for your child in 6 weeks”).

0.13.2 Case B - simulated focus-group: would parents pay EUR 297?

Without the system: the founder assumes yes, builds 8 weeks of content, launches, and meets the “no” at an empty cart - after burning months.

With the system: five diverse parent personas (grounded in real frustrations, budgets, alternatives, scepticism) were interrogated for friction, not applause. **Verdict: the hypothesis does not hold as stated.** The barrier is not price but *clinical credibility*, *time load*, and *proof of results*. Representative voices: Ahmed - “I’ve paid for promises three times”; Laura - “the problem isn’t information, it’s time and energy - another commitment sinks me”; Giulia - “I’d pay more for a real community, not to feel alone.” Real willingness-to-pay: ~2 of 5 at EUR 297 upfront. Kill-signals: upfront 8-week payment, no real physiotherapists, no measurable outcome, rigid live format, “motivational” positioning. Actions: tiered/monthly pricing; lead with clinical credentials and a data-backed case; sell *community + on-demand* rather than “coaching”; validate with 12-15 real interviews before building.

0.13.3 What the two cases show together

Run independently, premortem and focus-group **converged** on the same core insight - the risk is not the price but credibility, time and proof - from two different methods. That convergence is itself a validation signal. Both delivered, in minutes and before any code, exactly the objections a gut-feel launch would have found only after spending the money. Honest caveat: the focus-group is a *simulation* - it orients the questions and exposes blind spots; it does not replace 12-15 real interviews or measure true conversion. The system's value is not to *replace* evidence but to make you seek the right evidence, cheaply, first.

0.14 10. Limitations

- **Local embedding just achieved, not yet consolidated.** After the patch (§5.2) the embedder is local (bge-m3 on GPU), and the full corpus re-embed completed; the multilingual recall *quality* is content-limited (the vault holds mostly Microsoft career profiles, little stroke/disability content). The patch lives in a local fork of gbrain and must be re-applied after an engine update.
 - **Built != active-by-default (learned mid-session).** A recurring failure mode: mistaking “committed to the repo” for “active in the live environment”. Skills had to be *installed* to `~/.claude/skills/` and capture *enabled* (`RDA_LEARN=1`) for the “by default” promise to hold — done, but only after the gap was surfaced.
 - **Declared judgement seams.** `distill` writes `class: TODO` (no automatic classification) and evolve only fingerprint-diffs changelogs: both need an agent-in-the-loop. The design makes the gap harmless (curate rejects `TODO`; evolve is draft-only), but judgement is not automated — and should not be.
 - **Security must be tested, not assumed.** The audit showed the privacy invariant was a *textual promise + human gate*, not code, until implemented. Invariants of a self-referential system must be **tested**, because documentation can diverge from implementation.
 - **Residual sycophancy.** The focus-group mitigates but does not eliminate simulated-persona agreeableness; it is a tool for discovering questions and friction, not a substitute for real users.
 - **Single-user, single-machine.** Calibrated to one individual and one machine; cross-platform portability is designed but verified primarily on Claude Code.
 - **Self-modification risk.** A system that proposes changes to itself needs mechanical invariants (draft-only, scoped git-add, human gates, deny-list); their robustness is an assumption to be monitored continuously, not an acquired fact.
-

0.15 11. Discussion: principles that generalise

1. **Local-first for personal memory.** Privacy, zero-cost and availability outweigh a hosted model's marginal extra quality — especially in multilingual, sensitive contexts.
2. **Self-proposing, never self-applying.** Useful autonomy stops before the irreversible; value lies in *proposing with evidence*, not in acting alone over behaviour.
3. **Reuse > reinvention.** The temptation to build new structure (an ontology, an engine) often adds maintenance surface with no ROI; extending what exists is almost always better.
4. **Discovery before solving.** The highest value is not solving better, but choosing the right problem — which requires the user's voice and stress-testing the failure *before* the build.

5. **Verification over trust; honesty about failure.** The session’s most valuable safeguard was an internal audit that measured — it caught a false claim and a critical bug. A system that documents its own failures (a wiped brain recovered, mislabelled vectors corrected) is more trustworthy than one that reports only success.
-

o.16 12. Future work

- **Consolidate local embedding:** measure post-migration multilingual recall quality, and propose the patch upstream to gbrain (register bge-m3@1024) so a local fork is unnecessary.
- Verify real portability on Copilot and Codex (not merely designed).
- Automatic capture from session transcripts (today agent-driven).
- Focus-group panels calibrated on real audiences with consent.
- Longitudinal meta-loop metrics (how many promoted learnings survive review).
- Quantitative evaluation: measured recall/precision on a labelled query set; ablations of the meta-loop components.

o.17 13. Conclusion

roberdan-os shows that an individual can be given a coherent *agentic operating system* across tools, with its own private, persistent memory, and an improvement cycle that never betrays human control. Not one more assistant, but a second brain with its hands on the right gates — Remy in the hat, with the right taste and the hands still human. The session that built it also demonstrated the method: build, then *audit adversarially*, correct honestly, and keep the irreversible under human judgement.

o.17.1 Reproducibility and artefacts

Code and canon: git repository roberdan-os (private remote `github.com/Roberdan/roberdan-os`, ~24 commits as of 1 July 2026), entirely in English. Memory: Obsidian vault (local-first) + gbrain (local Postgres/pgvector); embedding **local** `ollama:bge-m3@1024` (via a patch to gbrain’s Ollama recipe, commit `f7376b11`). Scheduling: `launchd`. Governance: durable goal-ledger kanban. The optional heavyweight orchestrator (Convergio) was retired (idle, reversible). Canon, memory, orchestration **and** embedding are local-first — no cloud service or API key for core operation. Empirical checks and audit reproducible via `test/validate.sh`, `ollama ps`, and `gbrain` queries.